Resonance Guard – Technical Design Document (TDD)

1. Technical Overview

Engine

- **Godot 4.x**
- Language: **GDScript**

Architecture Principles

- **Composition over Inheritance**
- **Data-Driven Design (Resources)**
- **Decoupled Systems via SignalBus**
- **Single Responsibility per Component**

2. Scene Composition & Node Trees

2.1 Player Scene (`Player.tscn`)

Base Node: `CharacterBody2D`

```
Player (CharacterBody2D)
├── CollisionShape2D
├── Sprite2D
├── AnimationPlayer
├── MovementComponent (Node)
├── SquishComponent (Node)
├── ExpressionManager (Node)
```

Component Breakdown

MovementComponent.gd

Responsibility:

- Handle 2D movement
- Implement "Waddle" motion (oscillation-based)

Key Features:

- Uses velocity on both X and Y axes
- Applies oscillation (±10° feel via timing)

```
@export var move_speed: float = 250.0
@export var waddle_frequency: float = 0.4

var time := 0.0

func _physics_process(delta):
	time += delta
	var input_dir = Input.get_vector("left", "right", "up", "

	var wobble = sin(time / waddle_frequency) * 0.1

	var velocity = input_dir * move_speed
	velocity.x += wobble * move_speed

	get_parent().velocity = velocity
	get_parent().move_and_slide()
```

SquishComponent.gd

Responsibility:

- Handle squash/stretch feedback using **Tweens**

```
func squish():
	var tween = create_tween()
	tween.tween_property(get_parent(), "scale:x", 1.2, 0.1)
	tween.tween_property(get_parent(), "scale:x", 1.0, 0.1)
```

ExpressionManager.gd

Responsibility:

- Manage facial overlays / sprite expressions

Approach:

- Uses SpriteFrames or Atlas regions
- Switch based on state:
  - Idle
  - Stunned
  - Impact

2.2 Base Object Scene (`BaseObject.tscn`)

Base Node: `RigidBody2D`

```
BaseObject (RigidBody2D)
├── CollisionShape2D
├── Sprite2D
├── AudioStreamPlayer2D
```

BaseObject.gd

Responsibility:

- Apply data from `ShapeData`
- Emit signals on events

```
@export var data: ShapeData

func _ready():
	mass = data.mass
	physics_material_override = PhysicsMaterial.new()
	physics_material_override.friction = data.friction
	physics_material_override.bounce = data.bounciness

	$Sprite2D.texture = data.texture
```

3. Data-Driven Systems (Resources)

3.1 ShapeData.gd

```
extends Resource
class_name ShapeData

@export var mass: float
@export var friction: float
@export var bounciness: float
@export var impact_sound: AudioStream
@export var texture: Texture2D
```

Usage

- Create `.tres` files:
  - `wood.tres`
  - `stone.tres`
  - `rubber.tres`

3.2 WaveData.gd

```
extends Resource
class_name WaveData

@export var duration: float
@export var spawn_rate: float
@export var spawn_table: Dictionary
@export var win_condition_count: int
```

Example `spawn_table`

```
{
        "wood": 0.6,
        "rubber": 0.3,
        "stone": 0.1
}
```

4. Physics & Mechanics Logic

4.1 The "Sweeper" Mechanic

Goal:

- Player pushes objects only along **X-axis**
- Full freedom in **Y-axis** for positioning

Implementation Approach

Inside `Player`:

```
func _physics_process(delta):
        for body in get_slide_collision_count():
                var collision = get_slide_collision(body)
                if collision.get_collider() is RigidBody2D:
                        var rb = collision.get_collider()

                        var push_force = Vector2(velocity.x *
                        rb.apply_central_impulse(push_force)
```

Key Notes:

- Only **X-axis force applied**

- Y-axis unaffected → preserves stacking realism
- Strength scaling modifies impulse multiplier

4.2 Pressure System

PressureManager (Singleton / Autoload)

```
PressureManager (Node)
```

Responsibilities:

- Track all active objects on roof
- Calculate:
  - Base Pressure
  - Impact Spikes
- Emit warnings

Variables

```
var current_pressure: float = 0.0
var pressure_limit: float = 50.0
var active_objects: Array = []
```

Base Pressure Calculation

```
func calculate_base_pressure():
        var total = 0.0
        for obj in active_objects:
                total += obj.mass
        return total
```

Impact Spike (on landing)

```
func on_shape_landed(obj, velocity):
        var impact = obj.mass * velocity.length()
        current_pressure += impact

        SignalBus.emit_signal("shape_landed", obj)
```

Stabilization

- Gradually decay spike into base pressure

5. Decoupling & Communication

5.1 SignalBus (Global Event System)

Autoload: `SignalBus.gd`

```
extends Node

signal shape_landed(shape)
signal shape_cleared(shape)
signal roof_cracked(level)
```

Usage Examples

From BaseObject:

```
SignalBus.emit_signal("shape_landed", self)
```

From Boundary System:

```
SignalBus.emit_signal("shape_cleared", self)
```

From PressureManager:

```
if current_pressure > threshold:
        SignalBus.emit_signal("roof_cracked", current_pressure)
```

Benefits:

- Systems do NOT reference each other directly
- UI, Audio, Gameplay remain decoupled

6. Asset Lifecycle

Boundary Rules

- Left boundary: **x < 20**
- Right boundary: **x > 620**

Despawn Logic (BaseObject.gd)

```
func _physics_process(delta):
        if global_position.x < 20 or global_position.x > 620:
                SignalBus.emit_signal("shape_cleared", self)
                queue_free()
```

Notes:

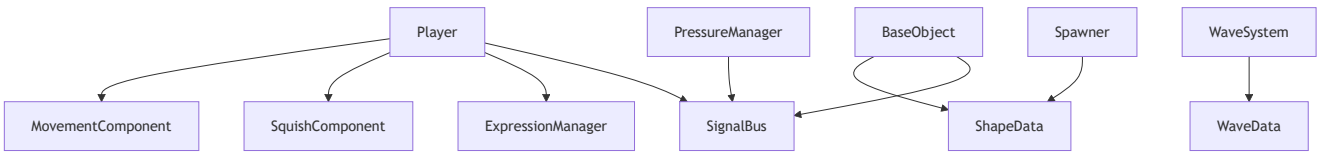
- Ensures memory cleanup
- Prevents physics buildup
- Updates pressure system via signal

7. System Flow

Object Lifecycle

```
Spawn → Fall → Land → (Impact Spike)
→ Settle → (Base Pressure)
→ Player Push → Exit → queue_free()
```

8. Class Diagram (Mermaid)



9. Godot 4 Implementation Notes

Tweens

- Use `create_tween()` (not Tween node)
- Chain animations for squish

Signals

- Use **SignalBus for global events**
- Avoid direct node references

Export Variables

- Use `@export` for:
  - Stats tuning
  - Resource assignment
  - Designer control

Physics

- Use `apply_central_impulse()` for pushing
- Use `RigidBody2D` for all objects

10. Final Notes

- All gameplay variations come from:

    - **ShapeData (materials)**
    - **WaveData (spawning)**
- No need for new object classes
- System is fully scalable and maintainable

Shortened version

Done. Copy to `TDD.md`.